

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Sameer Ahmed G	Reg. No.:	192421154
Section / Batch:	B Tech IT	Date:	06-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I Sameer Ahmed G (192421154) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sameer

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				

Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Intelligent_Vaccine_Cold_Storage

SIMATS Online Lab

Ask Gemini

Not secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=753

Search

Print

Download

School

More

SIMATS Online Programming Lab
DAA PROGRAMMING

SAMEER AHMED G
192A21104

ASSESSMENT-CO3/AT2-CASE STUDY

← Back

Language: Python C C++ Java

QUESTIONS

Q1
Scientific Computing (Matrix Multiplication)
100 marks

0/1 answered

Q1 Scientific Computing (Matrix Multiplication) 100 marks

A scientific application performs repeated matrix multiplications for simulations. Analyze how divide and conquer techniques can improve computational efficiency. Identify challenges in handling large matrices. Apply optimized matrix multiplication (such as Strassen's method). Analyze time complexity and compare with the traditional method. Design a solution and justify its advantages.

Your Code

```
1 def traditional_multiply(A, B):
2     n = len(A)
3     C = [[0] * n for _ in range(n)]
4     for i in range(n):
5         for j in range(n):
6             for k in range(n):
7                 C[i][j] += A[i][k] * B[k][j]
8     return C
9
10 def add_matrix(A, B):
11     n = len(A)
12     return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
13
14 def subtract_matrix(A, B):
15     n = len(A)
16     return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]
17
18 def split_matrix(M):
19     n = len(M)
20     mid = n // 2
21     A11 = [row[mid] for row in M[:mid]]
22     A12 = [row[mid] for row in M[mid:]]
23     A21 = [row[mid] for row in M[:mid]]
24     A22 = [row[mid] for row in M[mid:]]
25     return A11, A12, A21, A22
26
27 def combine_matrices(C11, C12, C21, C22):
28     top = [c11 + c12 for c11, c12 in zip(C11, C12)]
29     bottom = [c21 + c22 for c21, c22 in zip(C21, C22)]
30     return top + bottom
31
32 def pad_matrix(M):
33     n = len(M)
```

Input

stdin input

Output

Matrix A:
[9, 2, 2, 4]
[5, 6, 7, 8]
[9, 10, 11, 12]
[13, 14, 15, 16]

Scientific Computing (Matrix Multiplication)

A scientific application performs repeated matrix multiplications for simulations. Analyze how divide and conquer techniques can improve computational efficiency. Identify challenges in handling large matrices. Apply optimized matrix multiplication (such as Strassen's method). Analyze time complexity and compare with the traditional method. Design a solution and justify its advantages.

CASE STUDY: Scientific Computing using Divide and Conquer (Matrix Multiplication)

1. Case Understanding

Scientific computing applications such as weather forecasting, machine learning, image processing, and simulations perform matrix multiplication repeatedly. As the matrix size increases, the traditional multiplication algorithm becomes computationally expensive. Therefore, an efficient Divide and Conquer algorithm such as **Strassen's Matrix Multiplication** is used to reduce execution time and improve overall performance.

2. Key Issues Identified

- Large matrices require high computational time.
- Traditional multiplication performs many arithmetic operations.
- Memory usage increases with matrix size.
- Repeated matrix multiplication slows scientific simulations.
- Efficient algorithms are required to improve performance.

3. Why Divide and Conquer?

The Divide and Conquer technique divides a large matrix into four equal-sized submatrices. Each subproblem is solved independently and then combined to obtain the final result.

Advantages:

- Reduces computational complexity.
- Suitable for large matrices.
- Supports recursive implementation.
- Improves execution speed.

4. Traditional Matrix Multiplication

For matrices A and B,

$$\begin{bmatrix} C[i][j] = \sum_{k=1}^n A[i][k] \times B[k][j] \end{bmatrix}$$

Three nested loops are required.

Time Complexity:

$$\begin{bmatrix} O(n^3) \end{bmatrix}$$

5. Strassen's Matrix Multiplication

Instead of performing **8 recursive multiplications**, Strassen performs only **7 recursive multiplications**.

Matrices are divided into

```
[  
A=  
\begin{bmatrix}  
A_{11}&A_{12}\\  
A_{21}&A_{22}  
\end{bmatrix}  
\quad  
B=  
\begin{bmatrix}  
B_{11}&B_{12}\\  
B_{21}&B_{22}  
\end{bmatrix}
```

]

Seven products are calculated:

```
[  
P_1=(A_{11}+A_{22})(B_{11}+B_{22})
```

]

```
[  
P_2=(A_{21}+A_{22})B_{11}
```

]

```
[  
P_3=A_{11}(B_{12}-B_{22})
```

]

```
[  
P_4=A_{22}(B_{21}-B_{11})
```

]

```
[  
P_5=(A_{11}+A_{12})B_{22}
```

]

```
[  
P_6=(A_{21}-A_{11})(B_{11}+B_{12})
```

]

```
[  
P_7=(A_{12}-A_{22})(B_{21}+B_{22})
```

]

Final matrices:

```
[  
C_{11}=P_1+P_4-P_5+P_7
```

]

```
[  
C_{12}=P_3+P_5
```

]

```
[  
C_{21}=P_2+P_4
```

```

]
[
C_{22}=P_1-P_2+P_3+P_6
]

```

6. Sample Python Code

```

def multiply(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]

    return C

```

```

A = [[1,2],
      [3,4]]

```

```

B = [[5,6],
      [7,8]]

```

```

result = multiply(A,B)

```

```

print("Result Matrix")

```

```

for row in result:
    print(row)

```

7. Sample Output

Result Matrix

```

[19, 22]

```

```

[43, 50]

```

8. Comparison

Traditional Method	Strassen Method
Time Complexity = $O(n^3)$	Time Complexity = $O(n^{2.807})$
Uses 8 recursive multiplications	Uses only 7 recursive multiplications
Simple implementation	More efficient for large matrices
Best for small matrices	Best for large scientific computations

9. Time Complexity Analysis

Traditional Matrix Multiplication

[
 $O(n^3)$

]

Strassen Algorithm

Recurrence Relation

[
 $T(n) = 7T(n/2) + O(n^2)$

]

Using the Master Theorem,

[
 $T(n) = O(n^{\{2.807\}})$

]

10. Proposed Solution

The scientific application should implement **Strassen's Matrix Multiplication** for large matrices and use the traditional algorithm for smaller matrices. A hybrid approach minimizes execution time while maintaining accuracy and reducing computational overhead.

11. Justification

- Reduces the number of multiplication operations.
- Improves execution speed.
- Efficient for large datasets.
- Suitable for simulations and scientific computing.
- Supports recursive Divide and Conquer design.
- Reduces overall computational cost.

12. Conclusion

Scientific computing applications involve repeated matrix multiplication, making algorithm efficiency critical. While the traditional algorithm has a time complexity of $O(n^3)$, **Strassen's Divide and Conquer algorithm** reduces it to $O(n^{2.807})$ by decreasing the number of recursive multiplications. Therefore, Strassen's method is a more suitable choice for large-scale simulations, offering faster execution and better computational efficiency.